Klaster 1 — Kontrol Pembuatan Objek

Mata Kuliah:
CIF61123 – Pola Perancangan (3 SKS)
Pekan ke-7 | Sub-CPMK21 & 22
Disusun oleh:
Kelompok Pengelola Mata Kuliah Pola Perancangan



Versi Dokumen: 1.0
Tanggal: 17; Bulan: Agustus; Tahun: 2026
© Fakultas Ilmu Komputer, Universitas Brawijaya

filkom.ub.ac.id

Peta Perkuliahan

Kedudukan pertemuan ini di antara enam belas pekan perkuliahan

01	Konsep dasar & klasifikasi pola
02	Prinsip perancangan berorientasi objek
03	Interface, coupling–cohesion, UML
04	SOLID bagian 1 — SRP, OCP, LSP
05	SOLID bagian 2 — ISP, DIP
06	Code smell & refactoring
07	**Klaster 1 — kontrol pembuatan objek**
08	Ujian Tengah Semester
09	Klaster 2 — komposisi rekursif
10	Klaster 3 — pembungkus antarmuka
11	Klaster 4 — algoritma bervariasi
12	Klaster 5 — ketergantungan longgar
13	Katalog GoF & pedoman pemilihan
14	Studi kasus terpadu bagian 1
15	Studi kasus terpadu bagian 2
16	Ujian Akhir Semester

© Fakultas Ilmu Komputer, Universitas Brawijaya

SubCPMK yang Didukung

Pertemuan pertama yang membahas pola perancangan secara langsung

21 Sub-CPMK21

Mampu menerapkan pola Singleton dan Factory Method dalam bahasa pemrograman Java.

Bobot pada RPS: 7,50%.

22 Sub-CPMK22

Mampu menerapkan pola Abstract Factory serta menjelaskan maksud dan konsekuensi pola Builder dan Prototype.

Bobot pada RPS: 7,50%.

K1 Penilaian pekan ini

Sumatif — KUIS1 pada awal pertemuan.

Bobot: 10,00% melalui QUIZ1.

Mengukur Sub-CPMK11, 12, dan 13.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Materi Pembelajaran

Lima pokok bahasan, bersumber dari [1], [2], P1, dan P2

1 — Karakteristik creational

Masalah bersama yang dijawab kelima pola pembuatan objek.

2 — Factory Method

Menyerahkan keputusan jenis objek kepada subkelas.

3 — Singleton

Satu instans, beserta seluruh konsekuensi buruknya.

4 — Abstract Factory

Membuat keluarga objek yang saling berkaitan.

5 — Builder dan Prototype

Diperkenalkan ringkas sebagai pembanding.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Tujuan Pembelajaran

Setelah pertemuan ini mahasiswa diharapkan mampu:

- Menjelaskan karakteristik bersama creational pattern dan masalah perancangan yang dijawabnya.
- Menjelaskan struktur dan konsekuensi pola Factory Method serta mengimplementasikannya dalam Java.
- Menjelaskan struktur pola Singleton beserta konsekuensinya terhadap SRP, DIP, dan kemudahan pengujian.
- Membedakan penggunaan Factory Method dan Abstract Factory berdasarkan maksudnya.
- Menjelaskan maksud dan konsekuensi pola Builder dan Prototype.

© Fakultas Ilmu Komputer, Universitas Brawijaya

01

MATERI KE-1

Kuis 1 — Pelaksanaan pada Awal Pertemuan



Bagian 1 dari 6

© Fakultas Ilmu Komputer, Universitas Brawijaya

KUIS1 — Ketentuan Pelaksanaan

Materi pertemuan ke-1 sampai ke-6 — bobot total 10,00%

1 Cakupan

Sub-CPMK11 — 2,50%: konsep dasar dan klasifikasi pola.

Sub-CPMK12 — 2,50%: prinsip berorientasi objek dan pemodelan UML.

Sub-CPMK13 — 5,00%: prinsip SOLID, code smell, dan refactoring.

2 Bentuk soal

Pilihan ganda mengenai definisi, elemen penyusun, dan klasifikasi pola.

Uraian singkat: membaca class diagram atau potongan kode.

Analisis: menemukan pelanggaran SOLID dan mengusulkan teknik refactoring.

3 Aturan

Durasi 45 menit, dikerjakan pada awal pertemuan.

Bersifat tutup buku; perangkat elektronik tidak diperkenankan.

Kuis susulan paling lambat satu pekan setelah pelaksanaan.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Yang Diperiksa pada Lembar Jawaban

Pembahasan kunci jawaban dilakukan pada pertemuan yang sama sebagai umpan balik

- **Ketepatan istilah.** Nama pola, nama prinsip, dan nama code smell ditulis dengan benar.
- **Alasan, bukan sekadar vonis.** Menyebut “melanggar SRP” tanpa menyebut aktornya belum memperoleh nilai penuh.
- **Konsistensi diagram.** Notasi relasi dan arah panah pada class diagram harus benar.
- **Kelayakan usulan perbaikan.** Teknik refactoring yang diusulkan harus benar-benar menyelesaikan pelanggaran yang ditemukan.

◆ **Selanjutnya: setelah kuis, kita mulai membahas pola perancangan yang sesungguhnya.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

02

MATERI KE-2

Karakteristik Creational Pattern

Bagian 2 dari 6

© Fakultas Ilmu Komputer, Universitas Brawijaya

Masalah Bersama yang Dijawab Creational Pattern

Seluruh creational pattern menjawab satu pertanyaan: siapa yang boleh menyebut kata kunci new

Gejala pada kode

- **new tersebar di mana-mana.** Kode klien menyebut nama kelas konkret di banyak tempat.
- **Percabangan pemilihan jenis.** Rangkaian if–else untuk menentukan objek apa yang dibuat.
- **Keluarga objek tak terjaga.** Objek yang seharusnya berpasangan dapat tercampur tanpa terdeteksi.
- **Konstruktor berparameter panjang.** Sepuluh argumen yang sebagian besar boleh kosong.

Yang dilakukan creational pattern

- **Menyembunyikan jenis konkret.** Klien hanya mengenal tipe abstrak produknya.
- **Menyembunyikan cara pembuatan.** Keputusan objek apa yang dibuat dipindahkan ke satu tempat.
- **Menjaga konsistensi.** Objek satu keluarga dijamin dibuat bersama-sama.
- **Memisahkan perakitan dari hasil.** Proses menyusun objek dipisahkan dari bentuk akhirnya.

Intinya: creational pattern adalah penerapan Dependency Inversion pada persoalan pembuatan objek.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Lima Creational Pattern GoF

Tiga dibahas mendalam, dua diperkenalkan sebagai pembanding

Factory Method

Subkelas yang menentukan jenis objek yang dibuat.

Dibahas: sampai implementasi.

Abstract Factory

Membuat keluarga objek yang saling berkaitan.

Dibahas: sampai implementasi.

Singleton

Menjamin hanya ada satu instans.

Dibahas: sampai implementasi.

Builder

Memisahkan perakitan dari bentuk akhir objek.

Dibahas: ringkas, sebagai pembanding.

Prototype

Membuat objek baru dengan menyalin objek yang ada.

Dibahas: ringkas, sebagai pembanding.

© Fakultas Ilmu Komputer, Universitas Brawijaya

03

MATERI KE-3

Factory Method

Bagian 3 dari 6

© Fakultas Ilmu Komputer, Universitas Brawijaya

Motivasi: Percabangan Pemilihan Jenis Objek

Yang berubah dan yang tetap bercampur di dalam satu metode

TokoPizza.java

```
1   public class TokoPizza {
2       public Pizza pesan(String jenis) {
3           Pizza pizza;
4           if (jenis.equals("keju")) {
5               pizza = new PizzaKeju();
6           } else if (jenis.equals("sayur")) {
7               pizza = new PizzaSayur();
8           } else if (jenis.equals("kerang")) {
9               pizza = new PizzaKerang();      // jenis baru -> kelas ini diubah
10          }
11          pizza.siapkan();                    // bagian ini tidak pernah berubah
12          pizza.panggang();
13          pizza.potong();
14          return pizza;
15      }
16  }
```

- **Bagian yang berubah adalah pemilihan jenis; bagian yang tetap adalah urutan pengolahannya.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Factory Method — Maksud dan Peran

Salah satu pola yang paling sering muncul di dalam pustaka dan kerangka kerja

1 Maksud (intent)

Mendefinisikan antarmuka untuk membuat sebuah objek, tetapi membiarkan subkelas yang menentukan kelas mana yang diinstansiasi.

Dikenal juga sebagai: Virtual Constructor.

2 Peran yang terlibat

Product: antarmuka objek yang dibuat.

ConcreteProduct: implementasi nyata produk.

Creator: mendeklarasikan factory method dan memakai hasilnya.

ConcreteCreator: menentukan produk apa yang dibuat.

3 Kapan dipakai

Kelas tidak dapat mengetahui lebih dahulu jenis objek yang harus dibuatnya.

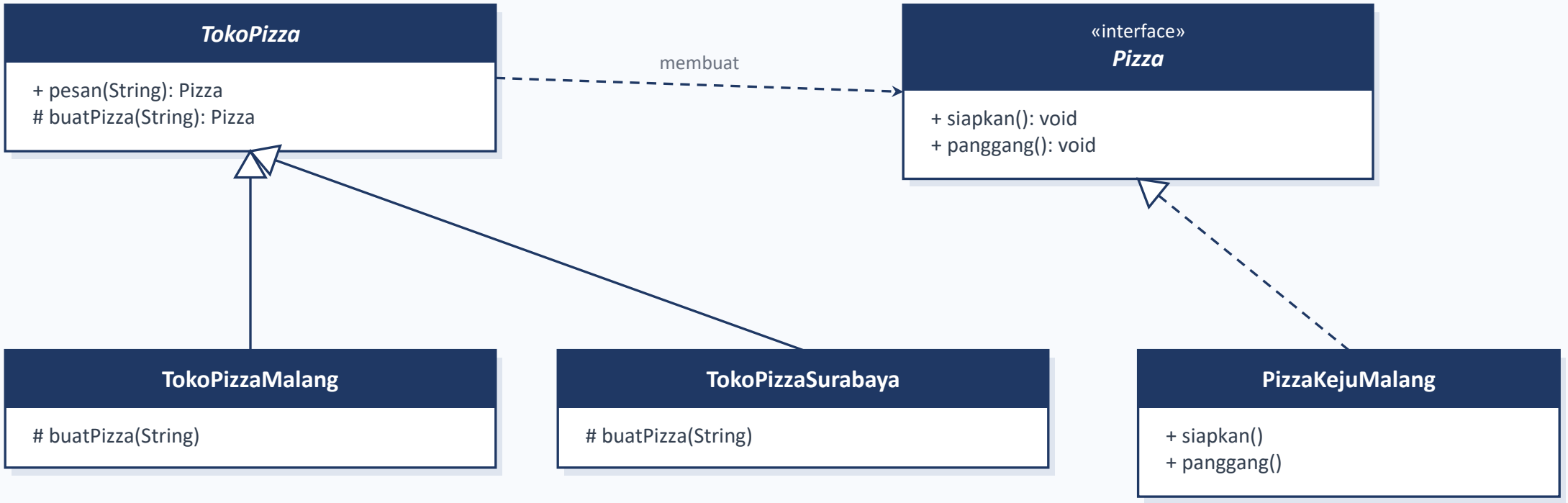
Keputusan jenis objek ingin didelegasikan kepada subkelas.

Kode klien perlu bebas dari nama kelas konkret.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Struktur Factory Method

Operasi buatPizza() adalah factory method-nya — abstrak pada Creator



- **Creator memakai produk tanpa pernah menyebut nama kelas konkretnya.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Implementasi Factory Method

Percabangan tidak hilang — ia dipindahkan ke satu tempat yang memang bertugas memilih

TokoPizza.java

```
public abstract class TokoPizza {

    // Template yang tidak pernah berubah.
    public Pizza pesan(String jenis) {
        Pizza pizza = buatPizza(jenis);      // <-- factory
method
        pizza.siapkan();
        pizza.panggang();
        pizza.potong();
        return pizza;
    }

    // Keputusan jenis objek diserahkan kepada subkelas.
    protected abstract Pizza buatPizza(String jenis);
}

public class TokoPizzaMalang extends TokoPizza {
    protected Pizza buatPizza(String jenis) {
        if (jenis.equals("keju")) return new PizzaKejuMalang();
        if (jenis.equals("sayur")) return new
PizzaSayurMalang();
        return null;
    }
}
```

- **Menambah cabang toko baru berarti menambah satu subkelas, bukan mengubah kelas TokoPizza.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Konsekuensi Factory Method

Pola ini memindahkan kerumitan, bukan menghapusnya — yang penting adalah ke mana ia dipindahkan

Yang diperoleh

- **Klien bebas dari kelas konkret.** Kode pemakai hanya mengenal tipe Pizza.
- **Memenuhi OCP.** Varian baru ditambahkan tanpa mengubah kelas Creator.
- **Titik perubahan tunggal.** Pemilihan jenis objek terkumpul di satu metode.
- **Hook bagi subkelas.** Subkelas dapat menyisipkan perilaku pembuatan yang berbeda.

Yang harus ditanggung

- **Subkelas bertambah.** Setiap varian Creator menuntut satu kelas baru.
- **Hierarki menjadi paralel.** Hierarki Creator berkembang mengikuti hierarki Product.
- **Perlu Creator abstrak.** Pola ini menuntut adanya kelas induk, sehingga kurang cocok bila hanya ada satu varian.

© Fakultas Ilmu Komputer, Universitas Brawijaya

04

MATERI KE-4

Singleton dan Konsekuensinya



Bagian 4 dari 6

© Fakultas Ilmu Komputer, Universitas Brawijaya

Singleton — Maksud dan Struktur

Pola paling pendek pada katalog GoF — dan yang paling sering disalahgunakan

1 Maksud (intent)

Menjamin sebuah kelas hanya memiliki satu instans, dan menyediakan satu titik akses global kepadanya.

2 Mekanismenya

Konstruktor dijadikan privat.

Instans disimpan pada atribut statis.

Akses hanya melalui metode statis getInstance().

3 Yang dijanjikan

Akses terkendali terhadap sumber daya yang memang tunggal.

Mengurangi pemakaian variabel global.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Tiga Cara Menulis Singleton dalam Java

Cara ketiga adalah yang dianjurkan Joshua Bloch dalam Effective Java

Konfigurasi.java

```
// 1. Lazy dengan penguncian – benar, tetapi setiap panggilan
terkunci.
public class Konfigurasi {
    private static Konfigurasi instans;
    private Konfigurasi() { }
    public static synchronized Konfigurasi getInstance() {
        if (instans == null) instans = new Konfigurasi();
        return instans;
    }
}
```

```
// 2. Initialization-on-demand holder – lazy sekaligus tanpa
penguncian.
public class Konfigurasi {
    private Konfigurasi() { }
    private static class Pemegang {
        static final Konfigurasi INSTANS = new Konfigurasi();
    }
    public static Konfigurasi getInstance() { return Pemegang.INSTANS;
}
}

// 3. Enum – paling ringkas, aman terhadap serialisasi dan refleksi.
public enum Konfigurasi { INSTANS; }
```

Ketiganya benar; yang berbeda hanyalah kinerja dan ketahanan terhadap penyalahgunaan.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Konsekuensi Buruk Singleton

Inilah sebabnya Singleton sering disebut anti-pattern yang menyamar sebagai pola

Terhadap prinsip SOLID

- **Melanggar SRP.** Kelas mengurus tugasnya sendiri sekaligus mengatur daur hidup instansnya.
- **Melanggar DIP.** Klien memanggil Konfigurasi.getInstance() — sebuah nama kelas konkret yang tertanam di kode.
- **Merusak OCP.** Mengganti implementasi menuntut kode klien diubah, karena tidak ada abstraksi di antaranya.

Terhadap pengujian dan pemeliharaan

- **Sulit diuji.** Instans tidak dapat diganti dengan objek tiruan pada pengujian.
- **Keadaan bocor antaruji.** Satu pengujian meninggalkan keadaan yang terbawa ke pengujian berikutnya.
- **Keadaan global tersamar.** Ketergantungan menjadi tersembunyi karena tidak muncul pada tanda tangan metode.
- **Bermasalah pada konkurensi.** Instans bersama menjadi titik rebutan antar-thread.

Intinya: Singleton menyelesaikan “berapa banyak instans”, tetapi menciptakan masalah “siapa yang bergantung pada siapa”.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Alternatif: Satu Instans tanpa Akses Global

Ketunggalan instans adalah urusan perakit, bukan urusan kelas itu sendiri

Singleton — ketergantungan tersembunyi

```
class LayananLaporan {
    void cetak() {
        var k = Konfigurasi.getInstance();
        // ketergantungan tidak terlihat
        // pada tanda tangan mana pun
    }
}

// Pengujian: instans tidak dapat diganti.
```

Injeksi — ketergantungan terbuka

```
class LayananLaporan {
    private final Konfigurasi konfigurasi;

    LayananLaporan(Konfigurasi k) {
        this.konfigurasi = k;
    }
    void cetak() { /* pakai konfigurasi */ }
}

// Perakit di luar menjamin hanya ada
// satu instans yang dibagikan.
```

◆ **Kolom kanan tetap memberi satu instans, tetapi ketergantungannya terlihat dan dapat diganti.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Kapan Singleton Masih Dapat Dibenarkan

Empat syarat — bila salah satu tidak terpenuhi, pertimbangkan kembali

1 Sumber daya yang memang tunggal

Kolam koneksi, pencatat log, atau berkas konfigurasi yang dibaca sekali.

2 Tanpa keadaan yang berubah

Objek yang bersifat tetap tidak menimbulkan kebocoran keadaan antaruji.

3 Diakses melalui abstraksi

Klien tetap bergantung pada antarmuka; ketunggalan diurus oleh perakit.

4 Bukan sekadar demi kemudahan

Memakai Singleton agar tidak perlu mengoper objek adalah alasan yang keliru.

© Fakultas Ilmu Komputer, Universitas Brawijaya

05

MATERI KE-5

Abstract Factory



Bagian 5 dari 6

© Fakultas Ilmu Komputer, Universitas Brawijaya

Ketika Objek Harus Dibuat Berpasangan

Inilah perbedaan mendasar antara Factory Method dan Abstract Factory

Masalahnya

- Sebuah pizza tersusun atas adonan, saus, dan keju.
- Cabang Malang memakai adonan tipis, saus tomat segar, dan keju mozarella.
- Cabang Surabaya memakai adonan tebal, saus pekat, dan keju cheddar.
- **Yang harus dijamin:** bahan dari dua cabang tidak boleh tercampur dalam satu pizza.

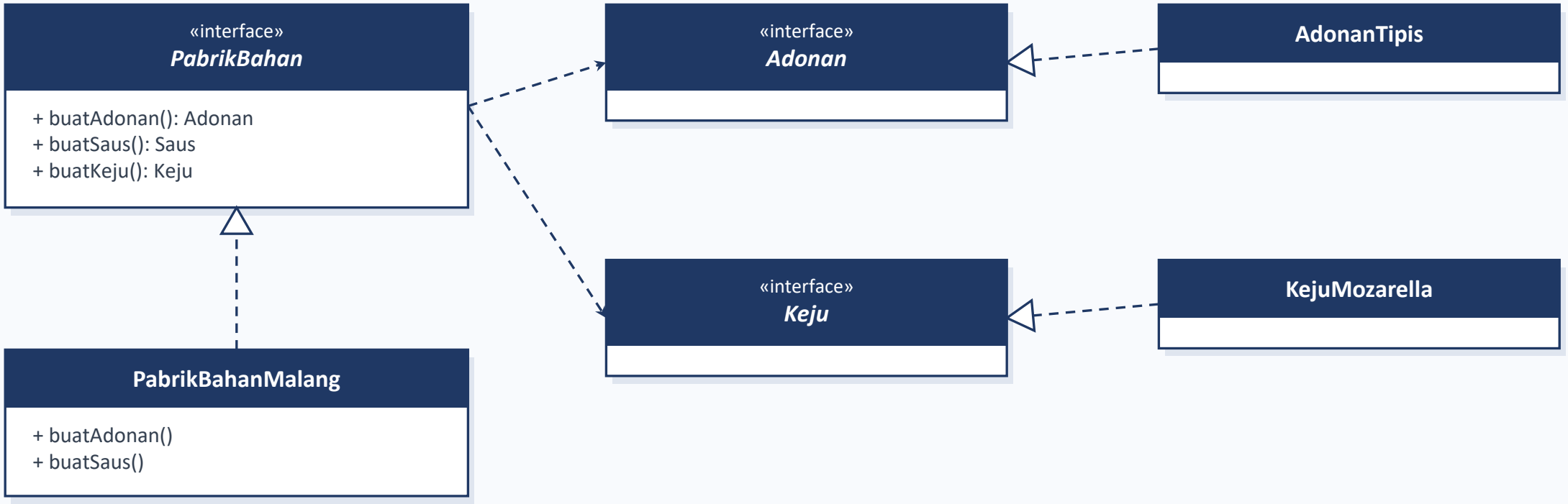
Mengapa Factory Method tidak cukup

- **Satu produk versus keluarga produk.** Factory Method membuat satu jenis objek; di sini ada enam jenis yang harus sepadan.
- **Konsistensi tidak terjaga.** Tidak ada yang mencegah adonan Malang dipasangkan dengan keju Surabaya.
- **Perlu satu titik penjamin.** Harus ada satu objek yang bertanggung jawab atas seluruh keluarga bahan.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Struktur Abstract Factory

Satu pabrik menghasilkan seluruh keluarga produk yang sepadan



Garis putus menyatakan realisasi antarmuka maupun ketergantungan pembuatan produk.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Implementasi Abstract Factory

Resep pizza tetap sama di semua cabang; yang berganti hanyalah pabrik bahannya

PabrikBahan.java

```
public interface PabrikBahan {
    Adonan buatAdonan();
    Saus   buatSaus();
    Keju   buatKeju();
}

public class PabrikBahanMalang implements PabrikBahan {
    public Adonan buatAdonan() { return new AdonanTipis(); }
    public Saus   buatSaus()   { return new SausTomatSegar(); }
    public Keju   buatKeju()   { return new KejuMozarella(); }
}
```

```
public class PizzaKeju extends Pizza {
    private final PabrikBahan pabrik;

    public PizzaKeju(PabrikBahan pabrik) { this.pabrik = pabrik; }

    public void siapkan() {
        adonan = pabrik.buatAdonan();     // seluruh bahan berasal
dari
        saus   = pabrik.buatSaus();       // satu keluarga yang sama
        keju   = pabrik.buatKeju();
    }
}
```

- **Mengganti satu objek pabrik mengganti seluruh keluarga bahan sekaligus.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Membedakan Factory Method dan Abstract Factory

Pertanyaan pada baris terakhir adalah yang paling menentukan pada ujian dan tugas

Aspek	Factory Method	Abstract Factory
Yang dibuat	Satu produk	Keluarga produk yang saling berkaitan
Mekanismenya	Pewarisan — subkelas menentukan	Komposisi — objek pabrik dikirim dari luar
Titik keputusan	Saat subkelas dipilih	Saat objek pabrik dipilih
Menambah produk baru	Mudah — tambah satu subkelas Creator	Sulit — seluruh antarmuka pabrik ikut berubah
Menambah keluarga baru	Tidak berlaku	Mudah — tambah satu kelas pabrik
Pertanyaan penentu	Siapa yang menentukan jenisnya?	Apakah objek harus sepadan satu sama lain?

Abstract Factory kerap diimplementasikan dengan Factory Method di dalamnya — keduanya bukan pilihan yang saling meniadakan.

© Fakultas Ilmu Komputer, Universitas Brawijaya

06

MATERI KE-6

Pengenalan Builder dan Prototype

Bagian 6 dari 6

© Fakultas Ilmu Komputer, Universitas Brawijaya

Builder — Memisahkan Perakitan dari Bentuk Akhir

Dibahas ringkas — tidak sampai tingkat implementasi

Maksud dan gejala

- **Maksud.** Memisahkan proses penyusunan objek rumit dari representasinya, sehingga proses yang sama dapat menghasilkan bentuk yang berbeda.
- **Gejalanya.** Konstruktor dengan sepuluh parameter yang sebagian besar boleh kosong.
- **Gejalanya.** Deretan konstruktor bertingkat (telescoping constructor) yang saling memanggil.

Konsekuensi

- **Kode pemanggil terbaca.** Setiap nilai disertai nama, sehingga urutan argumen tidak lagi menjadi jebakan.
- **Objek dapat dibuat tetap.** Seluruh nilai diketahui saat build() dipanggil, sehingga objek dapat bersifat immutable.
- **Kelas bertambah.** Satu kelas Builder untuk setiap kelas produk.
- **Berlebihan bila sederhana.** Objek dengan dua atau tiga atribut tidak memerlukannya.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Builder — Sekilas pada Tingkat Kode

Perhatikan keterbacaan sisi pemanggil, bukan sisi implementasinya

Peminjaman.java

```java
// Tanpa Builder: urutan argumen menjadi jebakan.
Peminjaman p = new Peminjaman(kode, ruang, mulai, selesai, null, null, false, 0);

// Dengan Builder: setiap nilai punya nama.
Peminjaman p = new Peminjaman.Builder(kode, ruang)
        .mulai(mulai)
        .selesai(selesai)
        .catatan("kuliah tamu")
        .butuhProyektor(true)
        .build();
```

- **Builder menukar tambahan satu kelas dengan kode pemanggil yang jauh lebih jelas.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Prototype — Membuat Objek dengan Menyalin

Dibahas ringkas — perbedaan salinan dangkal dan salinan dalam menjadi butir yang wajib dipahami

Maksud dan gejala

- **Maksud.** Menentukan jenis objek yang dibuat dengan memakai sebuah instans purwarupa, lalu membuat objek baru dengan menyalinnya.
- **Gejalanya.** Pembuatan objek jauh lebih mahal daripada penyalinannya.
- **Gejalanya.** Jenis objek yang harus dibuat baru diketahui ketika program berjalan.

Konsekuensi

- **Kelas konkret disembunyikan.** Klien menyalin objek tanpa mengenal kelas nyatanya.
- **Jenis baru saat berjalan.** Purwarupa dapat didaftarkan dan dicabut ketika program berjalan.
- **Bahaya salinan dangkal.** Salinan dangkal membuat dua objek berbagi objek anak yang sama.
- **Menyalin itu tidak sepele.** Setiap kelas harus benar-benar tahu cara menyalin dirinya.

Catatan: pada Java, antarmuka Cloneable dan metode clone() memiliki banyak jebakan; penyalinan lewat konstruktor salinan lebih dianjurkan.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Latihan di Kelas

Dua puluh menit berpasangan — bekal langsung bagi Tugas 3 pada pekan ke-10

1 Pilih polanya

Diberikan tiga pernyataan masalah pembuatan objek.

Tugas: tentukan Factory Method, Abstract Factory, atau bukan keduanya.

2 Bantah Singleton

Diberikan satu kelas yang memakai Singleton.

Tugas: sebutkan tiga konsekuensi buruknya dan tuliskan alternatifnya.

3 Implementasikan

Diberikan kerangka kelas dengan percabangan pemilihan jenis.

Tugas: ubah menjadi Factory Method dalam Java.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Simpulan

- Seluruh creational pattern menjawab satu pertanyaan yang sama: siapa yang boleh menyebut nama kelas konkret.
- Factory Method memakai pewarisan — subkelas Creator yang menentukan produk apa yang dibuat.
- Abstract Factory memakai komposisi — satu objek pabrik menjamin seluruh keluarga produk sepadan satu sama lain.
- Singleton menjamin ketunggalan instans, tetapi melanggar SRP dan DIP serta menyulitkan pengujian; ketunggalan sebaiknya diurus oleh perakit.
- Builder memisahkan perakitan dari bentuk akhir; Prototype membuat objek baru dengan menyalin purwarupa.

Pekan depan: **Ujian Tengah Semester pada pekan ke-8, mencakup Sub-CPMK11 sampai Sub-CPMK22 dengan bobot 30,00%.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Pustaka

Gaya pengacuan: Harvard-Anglia

- Gamma, E., Helm, R., Johnson, R. dan Vlissides, J., 1994. Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley, hlm. 87–134.
- Freeman, E., Robson, E., Bates, B. dan Sierra, K., 2004. Head First Design Patterns. Sebastopol, CA: O'Reilly Media, Bab 4 dan 5.
- Refactoring Guru, 2024. Creational Design Patterns. [daring] Tersedia pada: <https://refactoring.guru/design-patterns/creational-patterns> [Diakses 5 Juli 2024].
- Kuchana, P., 2004. Software Architecture Design Patterns in Java. Boca Raton, FL: CRC Press.

© Fakultas Ilmu Komputer, Universitas Brawijaya